第四部分

算法设计策略——贪心策略

主讲人：孙歧峰

联系电话：13864717907
Q Q号码：274342509
邮　　箱：sunqf@upc.edu.cn

算法设计策略——贪心策略（Greedy）

又称为优先策略，或登山策略。

我们来看一个找硬币的例子：

假设有四种硬币，它们的面值分别为**二角五分、一角、五分和一分**。现在要找给某顾客六角三分钱。给出找钱的方案。

显然，如果没有任何要求，会有很多方案。但如果我们要求用最少的硬币数，则得到解决方案就需要考虑了。

——这类问题在现实中很多！

怎么做？

4.2.1 最优化问题

现实世界中有一类问题，它可能有许多解，但是存在一个最优解的问题。——统称为最优化问题。 最优化问题（Optimization Problem），每个最优化问题都包含一组限制条件（Constraint）和一个优化函数（Optimization Function）。

符合限制条件的问题求解方案称为可行解（Feasible Solution）；满足约束条件的解不是唯一的！使优化函数取得最佳值的可行解称为最优解（Optimal Solution）；

根据最优化问题的定义，这类问题一般由两部分组成：

其一，一组限制条件（符合限制条件的解为可行解）

其二，一个优化函数（去优化可行解，从而得到最优解）



注意，优化函数又称为目标函数，一般在问题中给出，所谓最优解，就是该解使目标函数值达到最大（或 最小）。

算法设计策略——贪心策略（Greedy）

假定所有的可行解都属于一个侯选解集，若该侯选解集是有限集，从理论上可以使用穷举法（蛮力），逐一考察侯选解的每一个，检查它是否满足约束条件。若某个侯选解能够满足约束条件，则它便是一个可行解。此外，还可以同时用目标函数衡量——全局最优。 可行吗？

因此，对于有些问题，人们一直在研究、探索更快、更直接的求最佳解的方法。贪婪（贪心）策略就是其中之一。

一般地，如果一个问题适合用贪心法求解，问题的解应该可表示成一个*n*-元组$(x_0,x_1,\ldots x_{n-1})$，其中每个分量x_i取自某个值集S_i。所有允许的*n*-元组组成一个侯选解集。

x_0 x_1 x_2 … x_{n-1}

S_0 S_1 S_2 S_{n-1}

算法设计策略——贪心策略（Greedy）

贪心法是一种求解优化问题的算法设计策略。它是通过分步决策的方法来求解问题的，它在求解问题的每一步上做出某种决策，产生*n*-元组解的各个分量。

贪心法要求根据题意，选定一种最优量度标准，作为选择当前分量值的依据。

这种在贪心法每一步上用做决策依据的选择准则被称为**最优量度标准(Optimization criterion)或贪心准则(Greedy criterion)**，也称贪心选择性质。



但是这种度量标准通常只考虑局部最优性！

4.2.2 贪心策略的基本思想

贪心方法，顾名思义，是在决策中总是作出在当前看来是最好的选择，犹如登山一样，一步一步地向前推进。基本思想是：

- 求解最优化问题的算法包含一些列步骤；
- 每一步都有一组值可供选择；
- 在每一步上作出在当前看来最好的选择；
- 希望通过作出局部优化选择达到全局优化结果；

具体地：通过一系列的*选择*，得到问题的一个解，每个选择都是在当前某种意义（局部）下的最好选择即贪心选择，并且希望通过每次所做的贪心选择导致的最终结果是问题的一个最优解。

4.2.2 贪心策略的基本思想

在初始状态下，解向量$solution=\Phi$，其中未包含任何分量。使用最优量度标准，一次选择一个分量，逐步形成解向量$(x_0,x_1,\cdots,x_{n-1})$。在求解过程中生成的向量$(x_0,x_1,\cdots x_k)$ $k<n$，称为部分解向量，也称部分向量。

在根据最优量度标准选择分量的过程中，还要需要使用一个可行解判定函数。设$(x_0,x_1,\ldots,x_{k-1})$是贪心法已经生成的部分解，根据最优量度标准，来选取解向量的第$k(k<n)$个分量x_k，此时需要用可行解判定函数来判断在添加新的分量x_k后所形成的部分解$(x_0,x_1,\ldots,x_k)$是否违反可行解约束条件。若不违反，则$(x_0,x_1,\ldots,x_k)$构成新的部分解，并继续选择下一个分量，否则，需另选一个值作为分量x_k。

贪心法之所以被称为是贪心的，是因为它希望每一步决策都是正确的，即要求在求解的每一步上，仅根据最优量度标准选择分量，并只需保证形成的部分解不违反约束条件，最终得到的n-元组不仅是可行解，而且必定是最优解。

算法设计策略——贪心策略（Greedy）

但事实上，最优量度标准一般并不从整体考虑，它只是在某种意义上的局部最优选择，也就是说，每一步决策只是在当前看来是最优的，因此，贪心法不能保证对所有问题都得到整体最优解。

幸运的是对于很大范围内的问题，通过该方法使用精心考虑的选择准则，可以找到整体最优解；此外，在一些情况下，即使贪心法不能得到整体最优解，但其最终结果可能是最优的很好的近似解，所以该技术的应用很广！

例如：付款问题。

问题描述： 超市的自动柜员机（POS）要找给顾客各种数值的现金，表面上看，这是一种很简单的任务，但是交给机器办就不简单了。

人工？机器？

要注意，普通人在付款时都会自然地采用贪心的策略，不过人们往往并不在意所付的钞票数是否真的最少，实际上这个问题与不同钞票面值集合有关。例如，若面值为5元、4元、1元三种，要付8元，按贪心策略为为5+1+1+1共4张钞票，但实际上用两张4元的钞票即可！

4.2.3 贪心策略的基本要素

从上面分析我们知道，并不是所有的问题都可以用贪心策略来解决。那么，什么样的问题可以用贪心技术来求解呢？能否得到问题的最优解呢？这个问题很难给予肯定的回答。

但是，从大量的可以用贪心技术求解的问题可以看出，这类问题一般具有两个重要的性质，即应用贪心技术的基本要素：

- 最优子结构性质（Optimal substructure）
- 贪心选择性质（Greedy-choice-property）

4.2.3 贪心策略的基本要素

1. 最优子结构性质：

若问题的最优解包含着它的子问题的最优解，则称该问题具有最优子结构性质。

2. 贪心选择性质：

指问题的整体最优解可以通过一系列的局部最优的选择，即贪心选择来达到；这是贪心算法可行的第一要素！

贪心选择依赖于以往所做过的选择，而决不依赖于将来所作的选择，也不依赖于子问题的解。

由于贪心法并不能保证能找到最优的解，因此，用贪心法设计的求解问题的算法必须要证明其正确性，即设计的算法一定能得到问题的最优解。

4.2.3 贪心策略的基本要素

贪心策略设计算法的一般步骤应该包括两大步骤：

第一：分析问题，精心确定贪心选择准则 ，然后根据确定的选择准则设计出算法，进行局部优化选择；

第二：算法的正确性证明。

- 证明算法所求解的问题具有最优子结构性质；
- 证明算法所求解的问题具有Greedy选择性；

- 证明问题具有最优子结构性质：

一般采用反证法，即都是假设全局最优解如果不包含着子问题的最优解，那么通过假设的子问题的最优解就会推出我们原来假设的全局最优不正确（找出了比我们假设的最优解更优的解），从而矛盾！

4.2.3 贪心策略的基本要素

■ 证明问题具有贪心选择性质：

必须证明每一步所作的贪心选择最终导致问题的一个整体最优解。

首先证明问题的一个整体最优解，是从贪心选择开始的，而且，作了贪心选择后，原问题简化为一个规模更小的类似的子问题。然后用数学归纳法证明，通过每一步作贪心选择，最终可得到问题的一个整体最优解。

其中，证明贪心选择后的问题被简化为规模更小的类似问题的关键在于利用该问题的最优子结构性质。

4.2.4 贪心策略的基本步骤

（1）确定问题的可行解的求解方法，即满足约束条件的解；

（2）根据问题的贪心性质，确定贪心准则（局部最优的原则）；

（3）应用贪心准则，得到目前的局部最优解；

（4）简化原问题使之成为简单一些的子问题；

（5）自顶向下，以迭代的方式作出相继的贪心选择，直到问题的最优解得到。

（6）证明算法的正确性

算法设计策略——贪心策略（Greedy）

贪心策略算法框架：

```
Greedy(A, n)    // A[1:n]代表那些输入
   solution={};    //解向量初始化为空集
     for i from 1 to n do    //多步决策，每次选择解向量的一个分量
         x=Select(A);    //遵循最优量度标准选择分量
         //判定加入新分量后的部分解是否可行
         if Feasible(solution, x) then
              solution=Union(solution, x);    //形成新的部分解
         endif
     endfor
     return(solution)  // 返回生成的最优解
end Greedy
```

Select(A)是按照贪心准则选取A中的输入项；

Feasible(solution, x)是判断已知的解的部分solution与新选取的x的结合是否是可行解。

Union(solution, x)是形成新的部分解。

4.2.5 举例

例1：船的装载问题

问题描述： 有一条大船，最大载重为C，有n个重量不同，但大小相同的装货物的箱子，重量分别为$w_1,w_2,...w_n$。现要将箱子装船，要求在船上装入最多的箱子，且船不能超载。

问题分析：

这是一个最优化问题。

限制条件：$\sum_{i=1}^{n} xi.wi \le C$　　$xi=\begin{cases}1\\0\end{cases}$

可行解：满足限制条件的都是可行解；

最优解：$\sum_{i=1}^{n} xi$　最大

贪心准则是什么？

算法设计策略——贪心策略（Greedy）

算法：

```
Public static float laoding(float c, float[ ] w, int [ ] x)
{   int n=w.length;
    Element[ ] d=new Element[n];
    for(int i=0;i<n;i++)
        d[i]=new Element(w[i],i);   //物品的重量和编号
    MergeSort.mergeSort(d);   //排序，以便使用贪心准则
    float opt=0;
    for(int i=0;i<n;i++) x[i]=0;    //??
    for(int i=0;i<n&&d[i].w<=c;i++)    //??
        {  x[d[i].i]=1;  opt+=d[i].w;  c-=d[i].w;  }
    return  opt;
}
```

算法设计策略——贪心策略（Greedy）

证明：具有最优子结构性质

设$(x_1,x_2,\cdots,x_n)$是问题的一个最优解，则容易知道，$x_1=1$,且$(x_2,\cdots,x_n)$是船载重量为c-w_1，待装装箱子为$\{w_2,w_3,\cdots w_n\}$时相应的问题的最优解，即问题具有最优子结构性质。

反证法:假设 $(x_2,\cdots,x_n)$不是子问题的一个最优解。

不妨假设$(y_2,\cdots y_n)$是其一个最优解，我们构造一个原问题的新解：

$(x_1,y_2,\cdots y_n)$,我们知道，这是一个可行解，而且

$x_1+y_2+\cdots+y_n>x_1+x_2+\cdots+x_n$，这与$(x_1,x_2,\cdots x_n)$是最优解矛盾！

于是可得：

$(x_2,\cdots,x_n)$是船载重量为$c-w_1$，待装装箱子为$\{w_2,w_3,\cdots w_n\}$时相应的问题的最优解，即问题具有最优子结构性质。

算法设计策略——贪心策略（Greedy）

证明：证明满足贪心选择性质

设箱子已依其重量从小到大排序，$(x_1,x_2,\cdots,x_n)$是问题的一个最优解。又假设$k=min\{i,x_i=1\}$，易知，如果给定问题有解，则 $1\le k \le n$。

首先证明，一个整体最优解是从贪心选择开始的，即选择最轻的箱子。

（1） 当$k=1$时，显然成立。

$(x_1,x_2,\cdots,x_n)$是一个满足贪心选择性质的最优解。

（2）当$k>1$时，即假设整体最优解不是从贪心选择开始，我们证明一定能构造出另一个最优解，它是从贪心选择开始的。

构造（$y_1,y_2,\cdots y_n$），其中：$y_1=1,y_k=0,y_i=x_i,1<i\le n,i\ne k$,则

$$\sum w_iy_i=w_1-w_k+\sum w_ix_i\le\sum w_ix_i\le c$$

因此，$(y_1,y_2,\cdots y_n)$是所给问题的可行解。

另一方面，由 $\sum y_i=\sum x_i$知，$(y_1,y_2,\cdots y_n)$是满足贪心性质的最优解。

算法设计策略——贪心策略（Greedy）

再证明，证明作了贪心选择后，原问题被简化为一个更小的与原问题具有相同形式的子问题。

假设$(x_1,x_2,\cdots x_n)$是原问题的最优解，作了一次贪心选择后（$x_1=1$），原问题变为：载重为$C-w_1$,$n-1$个待装箱子，重量为$w_2,w_2,\cdots w_n$。要证明：$(x_2,x_3,\cdots x_n)$是这个子问题的最优解。

反证：假设该子问题存在更优的解，$(y_2,y_3,\cdots y_n)$ 则，$\sum y_i>\sum x_i\ i=2,3\cdots,n$

于是，我们可以构造出原问题的一个解：

$(z_1,z_2,z_3,\cdots z_n)$, 其中，$z_1=x_1$, $z_i=y_i, i=2,3,\cdots n;$

于是有：$\sum z_i>\sum x_i\ i=1,2,\cdots n$

这与$(x_1,x_2,\cdots x_n)$是最优解矛盾！

算法设计策略——贪心策略（Greedy）

例2：活动安排问题。

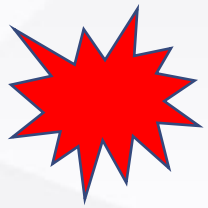
问题描述： 假设有n个活动的集合$E=\{a_1,a_2,...,a_n\}$，其每个活动都要求使用同一资源（如某个设备、教室、场地等），而在同一时间内只允许一个活动使用这一资源。每个活动都有一个要求使用该资源的起止时间s_i,f_i，且$s_i<f_i$。如果选择了活动a_i，则它在半开的时间区间$[s_i,f_i)$内占有资源。两个活动a_i,a_j称为是相容的，当且仅当它们的时间区间$[s_i,f_i)$和$[s_j,f_j)$不相交，即$s_i \geq f_j$ 或 $s_j \geq f_i$。现要求在所给定的活动集中选出最大的相容活动子集。

问题分析：这也是一个最优化问题。

限制条件：活动相容；

可行解：一组相容的活动；

最优解：最多的相容活动安排



那么，贪心准则是什么？

算法设计策略——贪心策略（Greedy）

贪心准则：

由于要安排尽可能多的相容活动，也就意味着必须为未安排的活动留下尽可能多的时间，所以贪心准则是：

使剩余的可安排时间极大化，即已经安排的活动尽可能占用少的时间。

贪心选择：每次选相容的、最早结束的活动。

（1）算法设计：

假设各个活动的起始时间和结束时间已经存储于一个二维数组中，且按活动的结束时间从小到大排序：

1	2	3
活动编号	起始时间	结束时间

算法设计策略——贪心策略（Greedy）

（2）算法：

```
n=待安排活动的个数;
A={data[1,1]};   /*第一个安排的活动 */
j=1;
for(i=2;i<=n;i++)
   if(data[i,2]>=data[j,3])  /* 相容活动、最早结束 */
     { A=A∪{data[i,1]};
       j=i;
     }
```

（3）算法分析：

在活动已经按结束时间排序的情况下，时间复杂性为O(n)，因此总的时间复杂性为O(nlogn)

算法设计策略——贪心策略（G

（4）举例

有11个活动，它们的开始和结束时

活　　动：	a_1	a_2	a_3	a_4	a_5	a_6
开始时间：	3	2	1	5	6	8
结束时间：	8	13	4	7	10	11

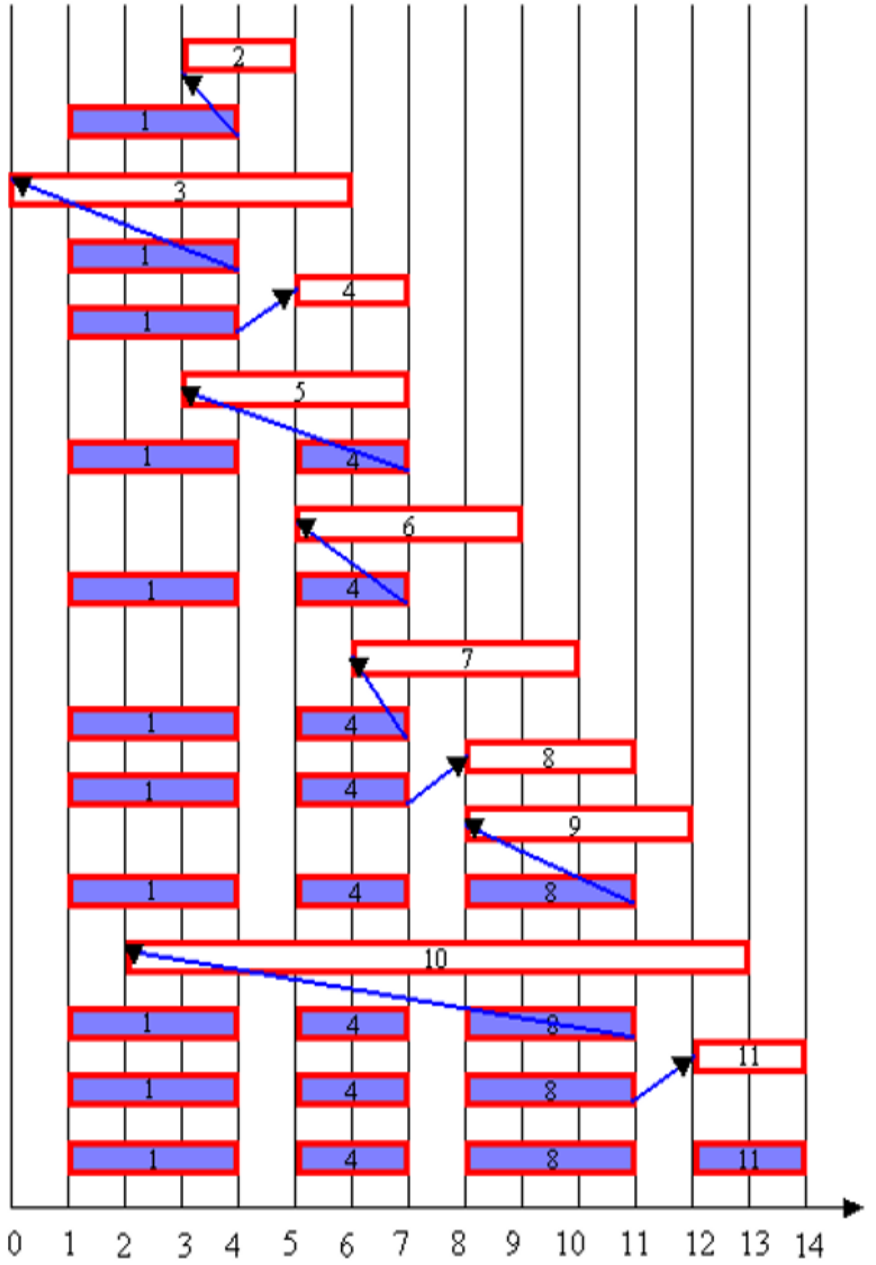
按活动结束的时间排序：

活　　动：	a_3	a_9	a_{10}	a_4	a_1	a_8
开始时间：	1	3	0	5	3	5
结束时间：	4	5	6	7	8	9

安排的活动：

$a_3 \longrightarrow a_4 \longrightarrow a_6 \longrightarrow a_7$

阴影长条表示的活动是已选入集合A的活动，而空白长条表示的活动是当前正在检查相容性的活动。



算法设计策略——贪心策略（Greedy）

（5）证明对于活动安排问题，这样的算法能得到最优解

假设E={a_1,a_2,...,a_n}是所给的活动的集合，且活动按结束时间非减的次序排列，即a_1是最早完成的活动。

第一步：证明活动安排问题有一个最优解以贪心选择开始，即有一个最优解中包含活动a_1。

假设A⊆E是活动安排问题的一个最优解，且A中的活动也按结束时间非减序排列，且A中的第一个活动是a_k。

若k=1，则A就是一个以贪心选择开始的最优解；

若k>1，则令B=(A-{a_k}) ∪ {a_1} ,由于$f_1 \le f_k$，且A中的活动是相容的，所以B中的活动也是相容的，且B中的活动个数与A中的活动个数相同（减一个、加一个），由于A是最优的，所以B也是最优的！所以总存在一个以贪心选择开始的最优活动安排。

算法设计策略——贪心策略（Greedy）

第二步：证明作了贪心选择后，原问题被简化为一个更小的与原问题具有相同形式的子问题。

设A是原问题的一个最优解，证明A’=A-{a_1}是对于E的活动子集E‘={$a_i \in E: s_i \geq f_1$}作活动安排的一个最优解。

若我们能找到E’的一个解B’，它包含比A’更多的活动，则将a_1加入到B‘中将产生E的一个解B，它包含比A更多的活动，于是，这与A的最优性矛盾。因此，每一步所作的贪心选择都将原问题简化为一个更小的与原问题具有相同形式的子问题。

对贪心选择的次数进行数学归纳即知，贪心算法最终产生原问题的一个最优解。

算法设计策略——贪心策略（Greedy）

从这两个例子可以看出，贪心策略的要素有：

- 适用于满足优化原则的组合优化问题
- 将问题表示成多步判断
  - 整个判断序列对应问题的最优解
  - 子序列对应子问题的最优解
- 确定一个优化测度（优化函数）
  - 不考虑以前各步的选择，只与当前状态有关
  - 以优化测度的极大(或极小)作为贪心选择的依据
- 确定是否满足贪心选择性质
  - 具有贪心选择性质得到最优解，否则为近似解

算法设计策略——贪心策略（Greedy）

■正确性证明

归纳法：证明贪心法得到最优解。对算法步数归纳、对问题规模归纳。

交换论证：在保证最优性不变的前提下，从一个最优解出发进行逐步替换，最终得到贪心法的解。

■自顶向下计算 – 通过贪心选择，将原问题规约为子问题。

贪心选择性质的证明

■ **方法---数学归纳法：**叙述一个可以归纳证明的命题：

●**对步数k归纳**

对于任意k，k 步贪心选择得到$i_1,i_2,\cdots,i_k$, 那么，存在最优解包含$i_1,i_2,\cdots,i_k$

●**规模k归纳**

对于任意*k*, 贪心法得到关于规模为k的问题的最优解

归纳基础：k=1, 命题为真

归纳步骤：假设对<k的正整数命题为真，证明对k命题 也为真(第二数学归纳法)。

算法设计策略——贪心策略（Greedy）

例4： 最优二叉树与*Huffman* 编码

Huffman编码技术有着广泛的应用背景（多媒体技术、计算机网络等），在数据压缩技术中是一种十分有效的方法。统计表明，对各种不同的数据文件，通过***Huffman***编码技术，可以节省20%—90%的空间。

信息的规范性以及能反映客观世界某些属性的内涵，很大程度体现在数据编码，或者说体现在数据编码规则和对规则的使用上。用少量、简单的基本符号，选用一定的组合规则，以表示大量复杂多样的信息就是编码。

编码的目的是将数据分类，规范整理为有效的信息。为此，现在绝大多数编码都是有信息含义的编码，对编码的要求一般体现在三个方面：

（1）唯一性（信息含义的唯一性）

（2）可分类

（3）反映所编码的承载信息，即数据内含的信息量。

算法设计策略——贪心策略（Greedy）

因此，编码实际上就是约定一个表示信息的规则。满足上面的三个要求。其中最基本的和最重要的一个要求就是“唯一性”！

一般来讲，人们还希望编码越简单越好，经过编码后的信息量不要膨胀，最好是变小（这就是压缩技术，因此压缩技术实际上就是研究编码）。因此，一套编码规则，在设计时要考虑下面两个方面：

首先，译码要唯一，确保唯一性！

其次，各个字符的编码要尽可能短，只有这样才能使编码后最短。

注意：要满足其一，很容易，但是要都达到就比较难了。

- *编码的方法：*
  - 定长编码方法—特点：规则简单，易达到译码唯一，但长度较大；我们常见的一些编码很多都是为定长编码，ASCII编码是定长编码。
  - 不定长编码方法—特点：长度较短，设计规则复杂，主要是因为要达到译码唯一较难。

算法设计策略——贪心策略（Greedy）

1. *Huffman*编码的基本思想

（1）解决“编码唯一”问题

Huffman编码采用“前缀码”来确保编码的唯一性。前缀码是一种字符编码方法，其任一字符的编码都不是其他字符编码的前缀。前缀码的译码过程因其前缀性质，不会产生二义性，确保了唯一性！

构造前缀码的方法是：以被编码字符作为叶子，构造出二叉树，然后一个左分支就有一位“0”码，一个右分支就有一位“1”码。于是从根到该字符的“0”、“1”序列就是其一个编码，而且一定是前缀码！ ——即用二叉树解决了前缀码问题，也就确保了唯一性。

我们可以知道，这样的编码会有很多，但是其编码长度肯定不同！

算法设计策略——贪心策略（Greedy）

（2）解决“最优（短）编码”问题

基本原则：文档中出现频率多的字符，编码应该尽可能短。体现在二叉树中就是该字符对应的叶子离根近（路径短）。所以，可以在构造二叉树时，考虑字符出现频率因素，让出现频率大的字符更靠近根（路径短）。这种树被称为最优前缀码二叉树（最优二叉树），*Huffman*给出了这种树的构造算法，因此又称为*Huffman* 树。

*Huffman*利用贪心策略设计出了构造最优二叉树的算法。

贪心准则：选择两棵根的权值最小的树，构造一棵新的树。

算法设计策略——贪心策略（Greedy）

2. *Huffman*树生成算法

设字符集为CS和频率表函数$f(c)(c \in CS)$已知，即CS中每个字符的频率值为$f(c)$。字符个数为n。$d_T(c)$ 表示字符c的深度（即路径长度）。

(1) 初始化。由n个单结点带权树组成集合(森林) Q:

$Q=\{ t(c) | c \in CS, W(t)=f(c) \}$ 森林中树的个数为 $m=n$

(2) 贪心选择。从Q中选取根权值最小的树t_1, t_2:

$$t_1 = \underset{W(t)}{MIN}\{ t \mid t \in Q \} \quad t_2 = \underset{W(t)}{MIN}\{ t \mid t \in Q-\{ t_1 \} \}$$

(3) 合并。以t_1,t_2分别作为左右子树，生成树 t，而且

$f(t)=f(t_1)+f(t_2)$,

以 t 取代森林中的 t_1, t_2, 森林中的树的树目m减1。

(4) 继续。如果m大于1，则继续转（2），否则，算法结束。

算法设计策略——贪心策略（Greedy）

3. 算法正确性证明

证明Huffman算法确实得到了最优前缀码二叉树（即前缀码问题的最优解）。分两步来证明：

证明算法的正确性，也就是证明如下两个命题是成立的。

命题7.4　两个最小权值字符对应的结点x和y必须是哈夫曼树中最深的两个结点且它们互为兄弟。

命题7.5　设T是字符集C对应的一棵哈夫曼树，结点x和y是兄弟，它们的双亲为z，显然有$w_z=w_x+w_y$，现删除结点x和y，让z变为叶子结点，那么这棵新树T_1一定是字符集$C_1=C-\{x, y\}\cup\{z\}$的最优树。

算法实现，见P276页。



算法设计策略——贪心策略（Greedy）

例5：单源点最短路径问题。

最短路径（shortest paths）是一种重要的图算法，在日常生活中的需求也非常普遍。

例如：

- 两地之间有无路可通？在有几条路的情况下，哪一条路最短？城市交通导游、GPS导游等等；
- 邮政自动分拣机的路选装置，要选择最短的路并投递；
- 计算机网络的路由选择尽管很复杂，但是最基本的路由问题仍然是最短路径问题

算法设计策略——贪心策略（Greedy）

定义最短路径，必须先定义路径权重的概念。给定一个带权重的图G=(V,E)，边的权重函数为w:E→R，则一条路径$p=v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_k$的权重为该路径上每条边的权重之和：

$$w(p)=\sum_{i=k}^{k-1} w(v_i, v_{i+1})$$

边的权重根据问题的不同，可以用长度、成本、时间等属性来度量。

无论是有向图还是无向图，从任意顶点u到顶点v的最短路径是从所有u到v的路径中权重之和最小的路径。最短路径权重定义为：

$$\delta(u,v)=\min\{\, w(p): p\text{是从}u\text{到}v\text{的路径}\}$$

说明：（1）并不是所有顶点之间都有最短路径，如果不连通，则不存在任何路径，也就谈不上最短路径。但是，也可以认为是有路径，只不过该路径的权重为无穷大。

（2）如果图包含一个权重为负的循环，则有些顶点之间将不存在最短路径。（为什么？）

算法设计策略——贪心策略（Greedy）

求任意“一对”顶点之间的最短路径及其路径权重是人们最容易想到的最短路径问题。这是所谓的“单源单点”最短路径问题，即只有一个始点，只有一个终点。

除此之外，还有其他的最短路径问题（好像不可理解，但是就是研究算法的人的非正常思维提出了该问题）：

（1）“单源多点”最短路径问题

（2）“多源多点”最短路径问题

算法设计策略——贪心策略（Greedy）

➢ 单源单点最短路径问题：

按照定义，要找一对顶点之间的最短路径，就要对它们之间存在的所有路径进行比较，而要找出一对顶点之间的所有路径，直观上，至少要走到过图的所有顶点，即按照某种方式对每个顶点进行考察，看看所求的最短路径是否需要经过当前被考察的顶点，如果有顶点没有被考察过，就无法判断当前找到的路径为最短路径，因为至少还有可能的路径没有考察过。因此，对图进行遍历应该是求单源单点最短路径的基础。

我们学习过遍历：

深度优先搜索(DFS)

广度优先搜索(BFS)

遍历顶点的时间成本均为$O(n)$，$n=|V|$，即顶点个数。

算法设计策略——贪心策略（Greedy）

但是，由深度或广度遍历求最短路径显然没有遍历这么简单，遍历是考察每个顶点一遍，但是寻找最短路径则至少考察每条边一遍（更多的情形是不止一遍，因为一条边可能包含在多条路径中），因此，其时间复杂性不会好于 $\Omega(n)$, $n=|E|$。

而且，直观感觉上，寻找单源单点最短路径的时间复杂性将大大超过边的数目！！

时间复杂性与一对顶点之间的路径的条数成正比，那么对顶点间有多少条路径呢？由图论知识我们可以知道：最坏情况下至少 $n=|V|$，即 $O(n!)$。 因此，最坏时间复杂性是巨大的！

对于一般性的一对顶点之间的最短路径问题，

用DFS或BFS求一对顶点之间的最短路径算法，请同学们自己尝试写出来

算法设计策略——贪心策略（Greedy）

➢ 单源多点最短路径问题：

尽管比较糟糕，但是我们发现，在找一对顶点之间的最短路径时，已经考察了源点到任意顶点的所有路径，既然如此，何不一次性地找出源点到所有顶点的最短路径呢？这样平摊下来，花费在一对顶点之间的最短路径上的成本就会大大降低！

更可喜的是，求一个顶点到其他所有顶点的最短路径问题可以使用贪心策略，贪心+摊销，从而使得最短路径的搜索成本大为下降。

1. 问题描述

给定加权的有向图G=(E,V)以及图中的一个顶点$s \in V$,求从s到其余各个顶点的最短路径。其中s称为源点。

注意：这里所指的路径长度是指路径上边所带的权值之和，而不是路径上边的数目，并假定边上的权值为非负值。

算法设计策略——贪心策略（Greedy）

2. 分析

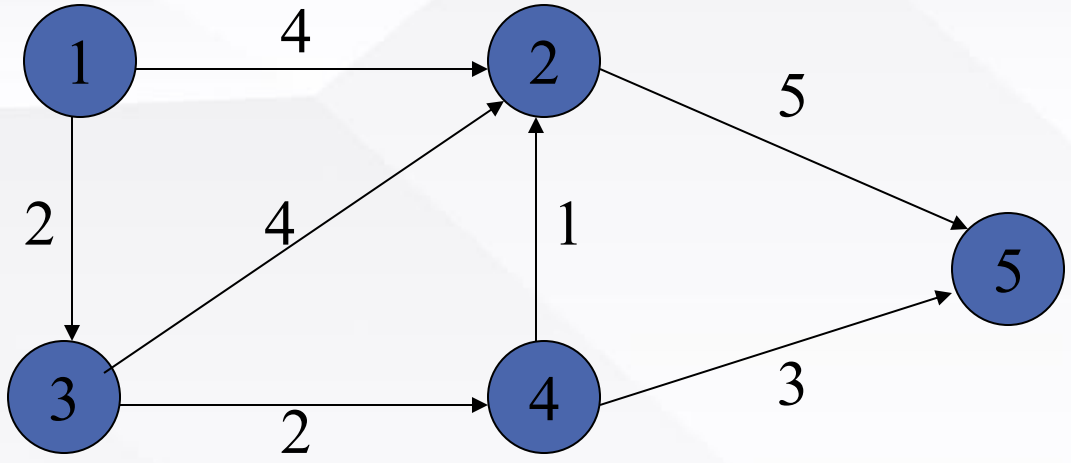
首先，求从顶点*u* 到顶点*v*最短路径(即单源单点最短路径问题)时，我们容易想到的一种求解的贪心策略是：从起点出发，每一步在路径中加入一个顶点（显然是有选择的）。

假设目前已经到达顶点*q*，但*q*不是目标顶点，下一步加入（走向）哪个顶点？　　—由贪婪准则确定！

简单的贪婪准则：选择离q最近的且目前还不在路径上的顶点。

采用这种贪婪准则的贪婪算法，很简单，看起来也好象正确，而且的确它有些时候能得到问题的最优解，但是它并不一定总能获得问题的最优解（最短路径）。例如，对下面的图：

算法设计策略——贪心策略（Greedy）



要找1到5的最短路径。

按照上面的算法：1——3——4——2——5　路径长度为10

显然不是最短的！！（更短的长度是7）

算法设计策略——贪心策略（Greedy）

算法设计策略——贪心策略（Greedy）

3. 复杂贪心求解—Dijkstra算法

从源顶点的角度看，路径往外延伸的下一个顶点就是目前离源点距离最近的顶点。为什么呢？

因为这个选择是一个最优选择：该顶点离源点的距离就是源点到该顶点的最短路径，即，我们做出这种选择后，就获得了源点到一个新顶点的最短路径！

于是，贪心准则可以定义为：

还没有求出的、离“源点”的距离最近的顶点。

从源点到另一个顶点的任何一条路径L_i均可以视为一个可行解，其中长度最短的路径就是从源点到该顶点的最短路径L_i^*。

从源点到其余每个顶点的最短路径构成了单源多点最短路径问题的最优解。

因此问题解的形式可以描述为：$L=(L_1,L_2,\cdots L_{n-1})$，只要每个分量$L_i$都是源点到某个顶点的路径，L就是一个可行解。

算法设计策略——贪心策略（Greedy）

根据上面的贪心准则，就有了“按路径长度的非递减次序逐一产生最短路径”的Dijkstra 算法的基本思想。

即，第一次应用贪心准则，求得长度最短的一条最短路径；

第二次应用贪心准则，再求得长度次短的一条最短路径；

其余类推，直到从源点到其他所有顶点之间的最短路径都已求得为止。

也就是说，对于最终求得的最优解$L^*=(L_1^*,L_2^*,\ldots,L_{n-1}^*)$，先求得其中最短的路径，然后再求次短的……。

算法设计策略——贪心策略（Greedy）

设s是源点，S={v_0,v_1,…,v_k}是已经求得最短路径的顶点结合，V-S是还未求出最短路径得顶点集合。一个顶点v_i属于 S当且仅当从源点s到v_i的最短路径已经求出。

于是，单源多点最短路径的*贪心准则*可具体描述为：

选取V-S中的顶点加入到S中，使得从s到S的所有顶点的路径长度之和增量最小。

即：总是在集合V-S中选择“当前最短路径”长度最小的顶点加入集合S中。“当前最短路径”：一个顶点t（t∈V-S)的当前最短路径，是一条从源点s到顶点t的路径，在该路径上，除顶点t外，其余顶点都属于S，当前最短路径是所有这些路径中的最短者。

Dijkstra算法就是按照这个贪心准则设计出的求最短路径的算法。算法描述如下：

假设源点为s，其他顶点为v_i。$w(i,j)$是有向边$v_i \rightarrow v_j$上的权值，$dist[i]$存放从源点s到顶点v_i的当前最短路径长度，设置一个顶点集合S，用于收集已求出的最短路径的顶点。用贪心准则来不断地扩充这个集合：一个顶点属于集合S当且仅当从源点到该顶点的最短路径长度已经求出。

算法设计策略——贪心策略（Greedy）

(1)初始化： 在初始状态下，集合S中只有一个源点s，即S={s}，所以

$$dist[i]=\begin{cases} w(s,i) & \text{若}\langle s,v_i\rangle\in E \\ \infty & \text{若}\langle s,v_i\rangle\notin E \end{cases}$$

*dist*中记录着s到各个顶点的当前最短路径长度。

(2) 求一条最短路径。（应用贪心准则选择求出每一步） 即在未求出最短路径的顶点中选取离源点距离最近的顶点。 在V-S中，选取具有最短的当前路径的顶点v_k，满足：$dist[k]=min\{\ dist[i]\ \mid\ v_i\in V-S\ \}$

(3) 将顶点v_k加入S中，对所有未求出最短路径且又与v_k邻接的顶点$v_j\in V-S$按下式修正（降距），即：$dist[j]=min\{dist[j],dist[k]+w(k,j)\}$

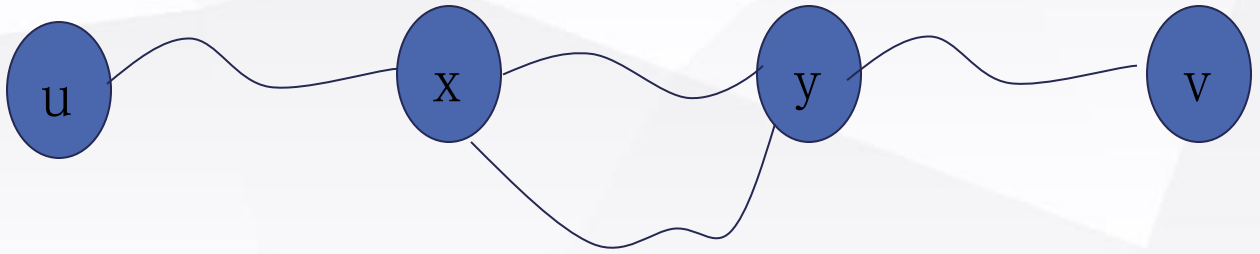
(4) 重复(2)、(3)，直到所有顶点最短路径都已经求出。

4. *Dijkstra*算法正确性证明

➢最优子结构性质： 顶点*u*,*v*之间的最短路径上的任何两个顶点*x*,*y*之间的路径也是相应的这两个顶点*x*,*y*之间的最短路径。(问题的最优解包含着子问题的最优解)

证明：反证法

如果 顶点*x*,*y*之间还存在一个更短的路径，则我们用这个更短的路径替换*u*,*v*路径上的*x*,*y*原来的这段路径，于是获得了一条更短的*u*,*v*之间的路径。而这与假设的*u*,*v*之间是最短路径矛盾！

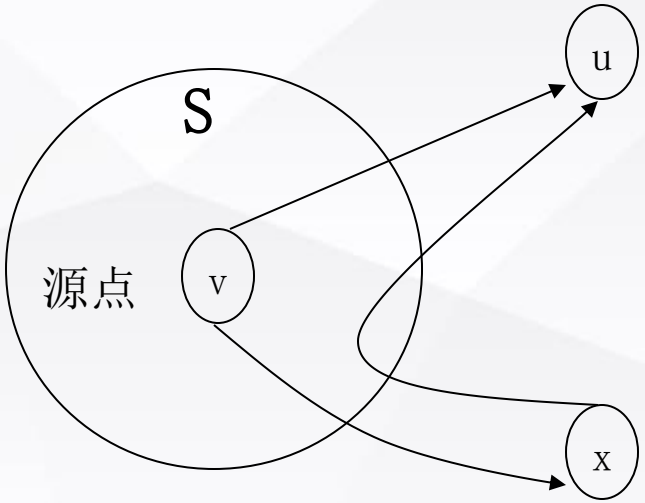


算法设计策略——贪心策略（Greedy）

➢贪心选择性质： 一次贪心选择是从V-S中选取具有最短当前路径的顶点*u*，从而确定了从源点到*u*的最短路径长度*dist[u]*。

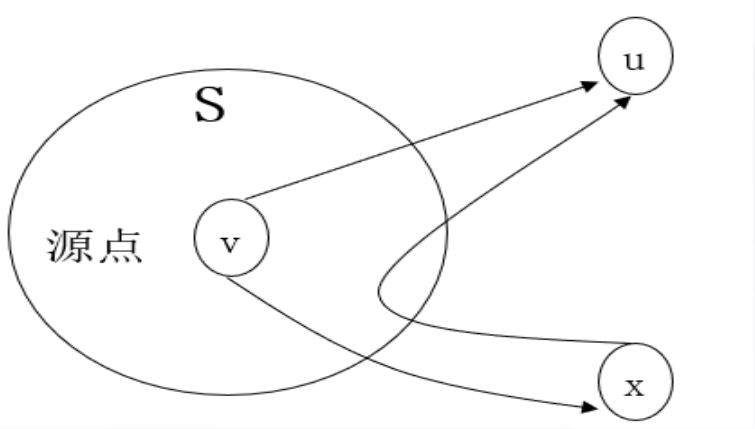
这种贪心选择为什么能导致最优解呢？即：为什么从源点到*u*没有更短的其他路径呢？

设源点为*v*，如果存在一条从源点到*u*且长度比*dist[u]*更短的路径，设这条路初次走出S之外到达的顶点为*x*∈V-S，然后徘徊于S内外若干次，最后离开S到达*u*。如下图：



算法设计策略——贪心策略（Greedy）

为了方便描述，我们定义：
d(v,x)=源点v到顶点x的路长
d(x,u)=顶点x到顶点u的路长
d(v,u)=源点v到顶点u的路长



根据我们的定义有：*dist[x] ≤ d(v,x)*

假设：　*d(v,x) + d(x,u) = d(v,u) < dist[u]*，　即存在更短的

而*d(x,u)*非负，可知　*d(x,u) ≥ 0*。则：

d(v,x) < dist[u],而　*dist[x] ≤ d(v,x)*，所以：

dist[x] < dist[u]，这与*x*在*S*外是矛盾的。

因此，贪心选择性质成立，即贪心选择后*dist[u]*就是从源点到顶点*u*的最短路径。

算法设计策略——贪心策略（Greedy）

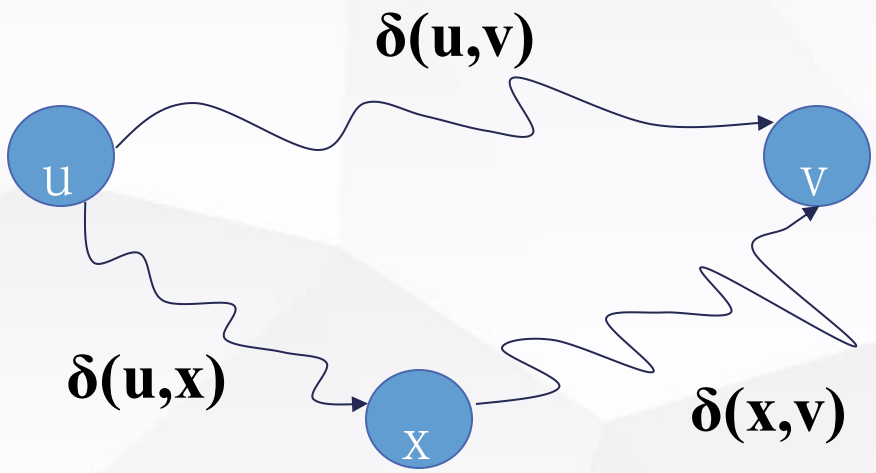
这个证明，实际上就是一个著名的三角不等式：

对于任意三个顶点$u,v,x \in V$, 有：

$$\delta(u,v) \leq \delta(u,x)+\delta(x,v)$$

$\delta(u,v)$ 是u到v的最短路径

反证法很容易证明！



应用该三角不等式，可以按前面的思路写出规范证明：

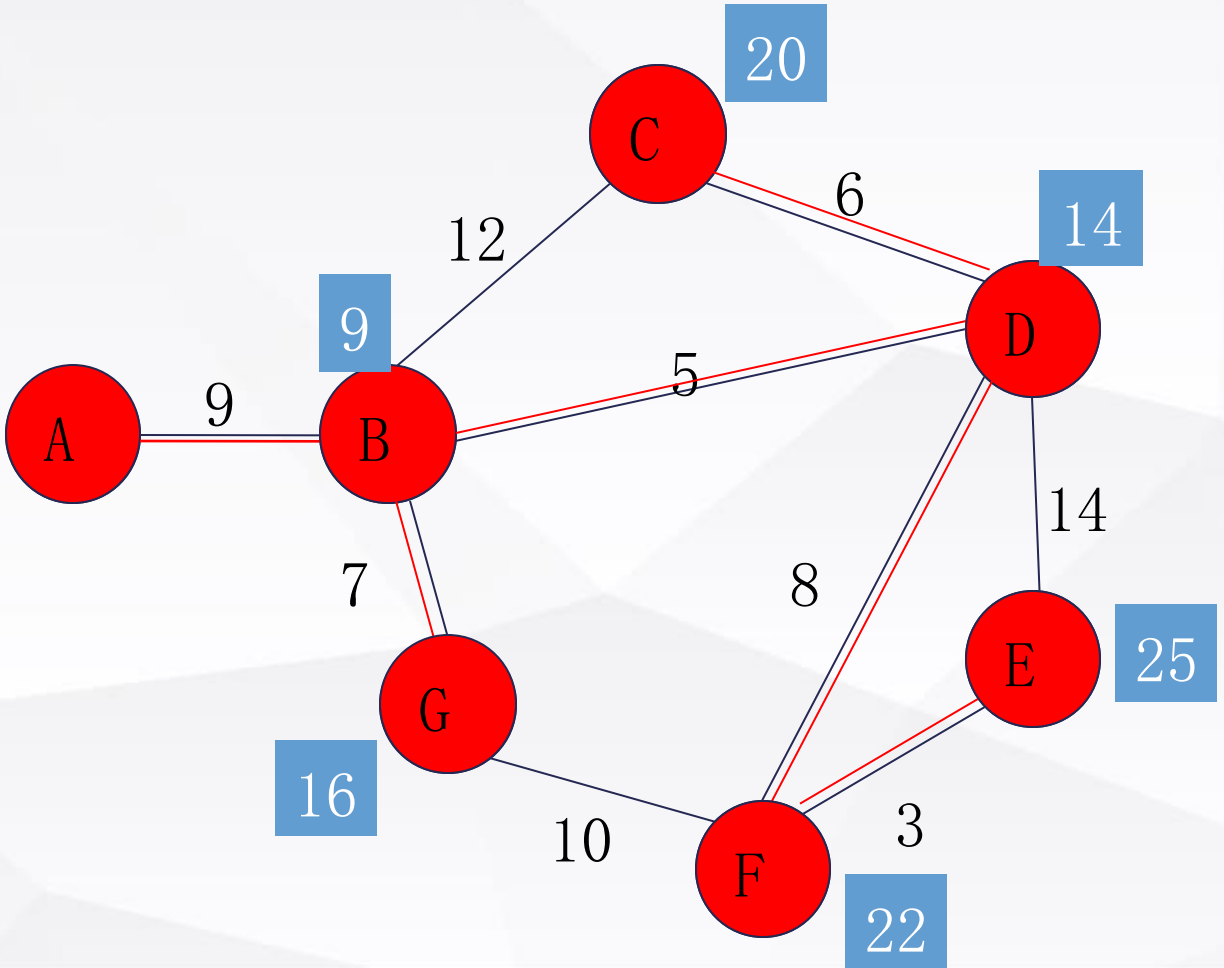
（1）从源点出发的可直达的最近的顶点第一个被求出！

（2）假设第$k-1$条已求出，第k条一定是当前路径最短的！

算法设计策略——贪心策略（Greedy）

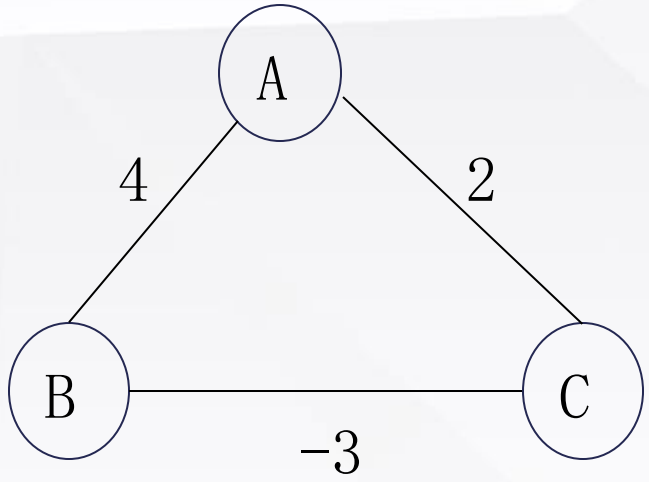
Dijkstra算法与洪水泛滥：

Dijkstra算法看起来比较难理解，但是可以发现，它和洪水泛滥极其相似。



算法设计策略——贪心策略（Greedy）

有负权重时 Dijkstra算法还行吗？



A到C的最短路径：

Dijkstra算法的结果A-C,2

实际上是A-B-C,1

原因：三角不等式不成立了！

在解决这个问题之前，我们先看看图中出现负路径有没有意义，如果没有意义的话，就不用操心这类问题了！

算法设计策略——贪心策略（Greedy）

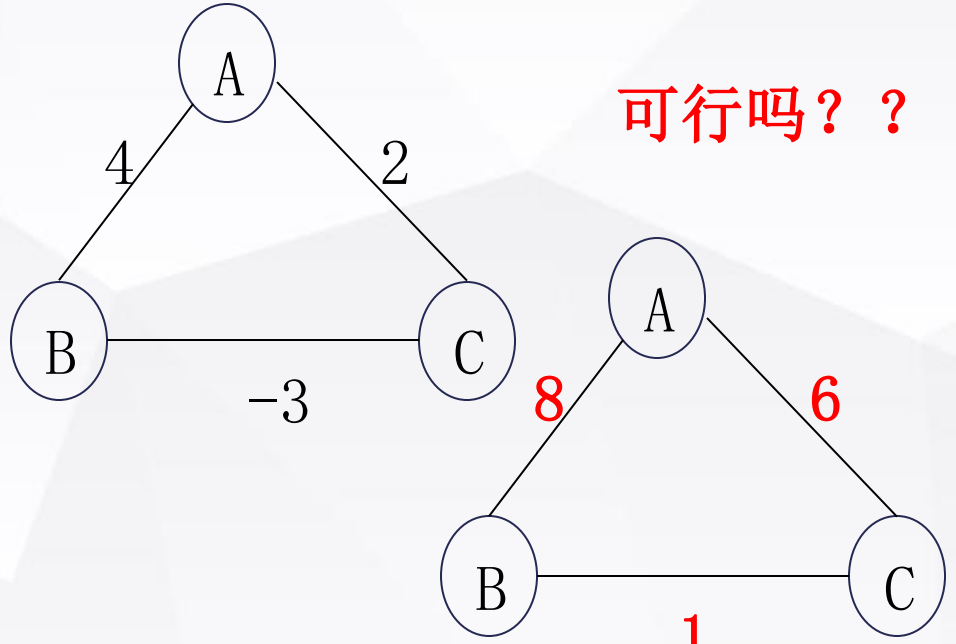
假设我们把路径上的收益作为权值，那么走这条路径因长度不同可能会付出不同的成本，但同时，也可能会在这条路上获得一些收入。那么，这样定义的路径权值就会出现负权值，即有些路径上只有付出，没有收获，或者收获小于付出。显然，这是有意义的！

物理上的时间隧道旅行，时间就是负数！即越旅行越年轻。

那么如何解决负权值的最短路径求解问题？

方法一：有人提出“等效变换”，基本思想是把具有负权值的图G转换为所有权值为正的图G’，然后再用Dijkstra算法。

例如，G中的最小权值为x, x<0。将G中的所有的边的权值减去x-1，则得到G’,权值全部为正。



方法二：Bellman-Ford算法

对Dijkstra算法进行改进，不再将所有顶点划分为两个集合，而是针对所有的边进行|V|轮降距操作。 因为，在三角不等式成立的情况下（权值全部为正），一个顶点加入到集合S，则其最短路径就求出来了，不再改变！但在有负权值边时，加入不加入到S中没区别，因为其最短路径都不能确定。

那么什么时候一个顶点的最短路径才能被最后确定呢？

在降距操作不再产生任何新的更短的路径长度时。——把针对顶点的降距改为对每条边进行降距。

动态规划思想

算法设计策略——贪心策略（Greedy）

例6：最小生成树

什么是生成树？

得到生成树的方法

什么是最小生成树？

一棵生成树的代价是树中各条边上的代价之和。具有最小代价的生成树称为最小代价生成树（Minimum Cost Spanning Tree）

1. 贪心策略求解的思想

可行解：生成树

最优解：最小生成树

蛮力法（穷举）：

贪心策略： 根据贪心策略的思想，关键问题是需要确定贪心准则，根据最优量度标准，每一步从图中选择一条符合贪心准则的边，共选择n-1条边就结束了。

算法设计策略——贪心策略（Greedy）

设G＝（V，E）是加权的连通图，n是顶点的数目。最小生成树T=(V,TE)。最小代价生成树的贪心策略算法可以根据我们贪心算法的一般框架写出来，如下：

```
Spanning(G,n)
{
  TE=Φ;
  k=0;
  while(k<n-1&&E中尚有未检查的边)
  { e=Select(E);    //按量度标准选择一条边
    if(可行)        //判断可行性，是生成树
     { TE=TE∪{e};k++;}
   }
 }
```

算法设计策略——贪心策略（Greedy）

最简单的量度标准是：选择使得迄今为止已入选TE中边的代价之和增量最小的边。基于这个度量标准可产生不同的构造最小代价生成树的贪心算法。

著名的PRIM（普里姆）算法和KRUSKAL（克鲁斯卡尔）就是其中两个典型的贪心算法。

Prim方法的贪婪准则：在保证T子图自然是一棵的情况下，从剩余的边中（一个顶点在树上，另一个不在），选择一条权值最小的边（增量最小），并且它的加入应使所有入选的边仍然是一棵树。

Kuskal方法的贪心准则：每次从剩余的边中选择一条不会产生回路（不在同一个连通分支上）的具有最小权值（增量最小）的边加入已经选择的边的集合中。（过程中T子图不一定连通）

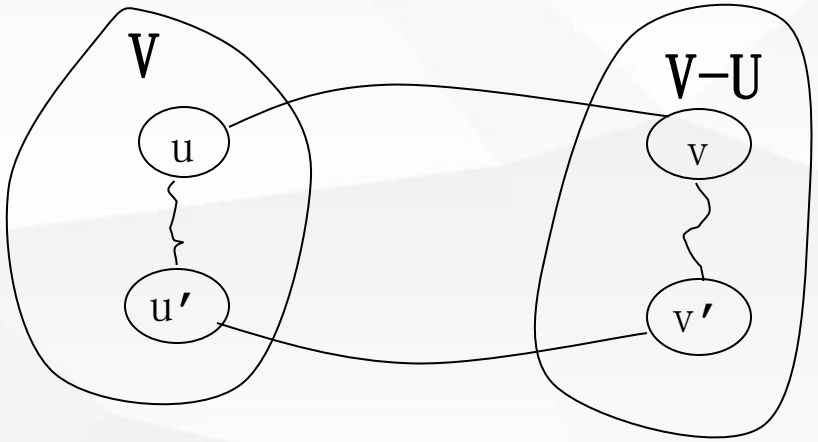
算法设计策略——贪心策略（Greedy）

2. 算法的正确性证明

***PRIM*和*KRUSKAL*算法的正确性都是建立在MST性质结论之上的。**

MST性质(贪心选择性质)：设图G=(V,E)是一个加权连通图，U是V的一个真子集。若边(u,v) ∈E是所有u ∈U，v ∈V-U的边中权值最小的，那么一定存在G的一棵最小代价生成树T=(V,TE)，(u,v) ∈TE。 ——“割性质”

证明：反证法。如果图G的任何一棵最小代价生成树都不包括(u,v)。我们将(u, v)加到图G的一棵最小代价生成树T中，将形成一条包含边(u, v)的回路，并且在此回路上必定存在另一条不同的边(u′, v′)，使得u′∈U，v′∈V-U。如下图所示：



删除边(u′, v′)，便可消除回路，并同时得到另一棵生成树T′。因为(u, v)的权值不高于(u′, v′)，即删除一条权值大的边，加入一条权值小的边，则T′的代价一定不高于T，且T′包含边(u, v)，故与假设矛盾。得证！

基于这个贪心性质，就也可以用归纳法证明两个算法的正确性。

算法设计策略——贪心策略（Greedy）

- Prim算法的正确性证明（对步数归纳）：

命题：对于任意k<n，存在一棵最小生成树包含算法前k步选择的边。

证明：*k=1*，存在一棵最小生成树T包含边*e={1,i}*，其中*{1,i}*是所有关联1的边中权最小的。反证：

设T为一棵最小生成树，假设T不包含*{1,i}*，则*T∪{{1,i}}*含有一条回路，回路中关联1的另一条边为*{1,j}*，

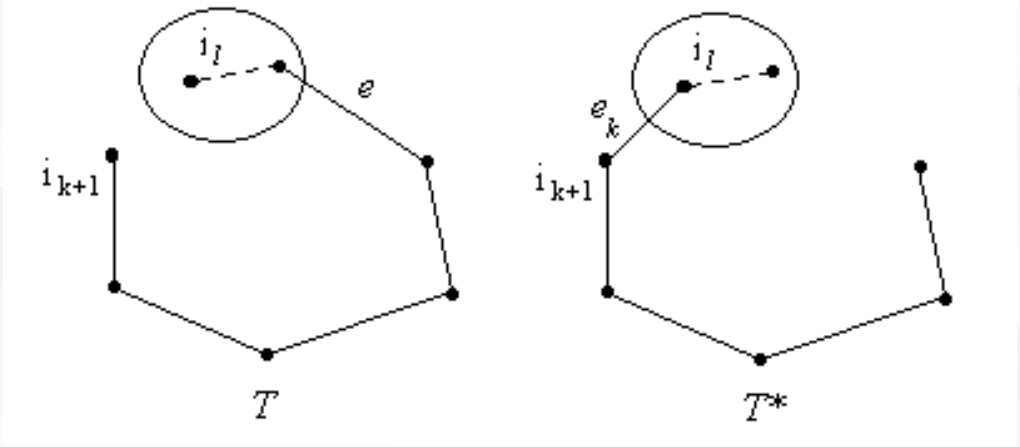
$$T' = (T - \{\{1,j\}\}) \cup \{\{1,i\}\}$$

也是生成树，且$W(T') \leq W(T)$。矛盾！

算法设计策略——贪心策略（Greedy）

假设算法进行了$k-1$步，生成树的边为 $e_1,e_2,\cdots,e_{k-1}$，这些边的k个顶点构成集合S. 由归纳假设存在G的一棵最小生成树T包含这些边.

算法第k步选择了顶点i_{k+1}，则i_{k+1}到S中顶点的边权最小，设这条边为$e_k=\{i_{k+1},i_l\}$。假设T不含有$\{i_{k+1},i_l\}$，则将e_k加到T中形成一条回路. 这条回路有另外一条连接S与$V-S$中顶点的边e，令$T^*=(T-\{e\})\cup\{e_k\}$，则T^*是G的一棵生成树，包含了$e_1,e_2,\cdots,e_k$，且$W(T^*)\le W(T)$。矛盾！



算法设计策略——贪心策略（Greedy）

➢ *Kruskal*算法的正确性证明：（对顶点数归纳）

命题：对于任意*n*，使用*Kruskal*算法对于*n*阶图得到一棵最小生成树。

证明 *n=2*，只有一条边，命题显然为真.

假设对于*n*个顶点的图算法正确；

现考虑*n+1*个顶点的图*G*，*G*中最小权边 *e = {i,j}*.

从*G*中短接*i*和*j*，得到图*G′*为*n*个顶点的图。根据归纳假设，由算法存在*G′*的最小生成树*T′*。

算法设计策略——贪心策略（Greedy）

令$T=T'\cup\{e\}$, 则T是关于G的最小生成树.

因为:若不然，存在一棵G的最小生成树T^*，$W(T^*)<W(T)$.

（1）如果 e 属于T^*, 则短接e得到G'的生成树$T''=T^*-\{e\}$,

且$W(T'')<W(T')$, 与T'的最优性矛盾；

(2)如果e不属于T^*, 在T^*中加上边e，形成回路, 去掉回路中任意别的边所得生成树的权小于$W(T^*)$，与$W(T^*)$的最优性矛盾。

思考：（1）算法实现中有哪些问题问题

（2）最小生成树与聚类

算法设计策略——贪心策略（Greedy）

例7：背包问题

问题描述： 已知一个载重为M的背包和n件物品，它们的重量分别为$w_1, w_2, \ldots, w_n$，价值分别为$p_1, p_2, \ldots, p_n$，现要求在背包中装入物品，使得背包不能超重，且装入的物品的价值最大。

所谓的背包问题，是指求一种最佳装载方案，使得价值最大。它是现实世界里一个常见的最优化问题。

背包问题有两类：

- 可分割背包问题：如果物品是可以分割的，即允许将一件物品的一部分装入背包。若装入第i件物品的一部分x_i $(0 \leqslant x_i \leqslant 1)$，则该物品的重量为$w_i x_i$，其价值为$p_i x_i$。
- 不可分割背包问题：每件物品不允许分割，只能作为整体或者装入，或者不装入。 ——0/1背包问题。

算法设计策略——贪心策略（Greedy）

1．可分割的背包问题

问题描述： 假设x_i是物体i被装入背包的部分， $0 \le x_i \le 1$，$x_i=0$时表示物体i没有被装入背包，当$x_i=1$时表示物品被全部装入背包。该问题的解可以表示成 $X=(x_1,x_2,\cdots,x_n)$解向量的形式。于是有问题的约束条件和目标函数：

约束条件：$\sum w_i x_i = M$　　$w_i>0,\ 0 \le x_i \le 1,\ 1 \le i \le n$

目标函数：$d = max \sum p_i x_i$　$p_i>0, 0 \le x_i \le 1,\ 1 \le i \le n$

算法设计策略——贪心策略（Greedy）

贪心准则的选择：

（1）价值优先。即首先选择价值大的装入背包。不一定能求得最优解，因为背包得容量可能会因为选中物体很重而很快消耗掉。

（2）重量优先。即首先选择重量小的装入背包（让背包的载重尽可能消耗的慢）。也不一定能求得最优解，因为物体重量小得到的价值也小。

（3）价值重量比优先。即首先选择单位重量价值高的物品装入背包（让背包的载重尽可能消耗的慢，而且获得的价值增加最快）。——可以求出最优解，正确的贪心准则。

算法自己写出，并证明正确性。

2. 0/1背包问题

问题描述： 设背包的容量为M，有n个物品，它们的重量分别为$w_1,w_2,\ldots,w_n$，价值分别为$p_1,p_2,\ldots,p_n$。现要求在背包中装入物品（不可分割），使得背包不能超重，且装入的物品的价值最大。

约束条件：$\sum w_i x_i \le M \quad x_i \in \{0,1\} \quad (1 \le i \le n)$

目标函数：$\sum p_i x_i$ 最大 $\quad x_i \in \{0,1\} \quad (1 \le i \le n)$

算法设计策略——贪心策略（Greedy）

贪心准则1： 每次选择价值最大的物品放入背包——价值贪心；

不一定能找到最优解（因为容量可能很快用完）；

自己举例说明。

贪心准则2： 每次选择重量最小的物品放入背包——重量贪心；

也不一定能找出最优解；

自己举例说明。

贪心准则3： 每次选择p_i/w_i最大的物品放入背包； 也不一定！

自己举例说明。

算法设计策略——贪心策略（Greedy）

我们不必因所考察的几个贪婪算法都不能保证得到最优解而沮丧， 0/1背包问题是一个NP-复杂问题。对于这类问题，也许根本就不可能找到具有多项式时间的算法。

虽然按p_i/w_i 非递（增）减的次序装入物品不能保证得到最优解，但它是一个直觉上近似的解。我们希望它是一个好的启发式算法，且大多数时候能很好地接近最好算法。

实验表明：在600个随机产生的背包问题中，用这种启发式贪婪算法来解有239题为最优解。有583个例子与最优解相差10%，所有600个答案与最优解之差全在25%以内。该算法能在$O(n\log n)$时间内获得如此好的性能。

思考： 该实验怎么做？ 有哪些难点？

算法设计策略——贪心策略（Greedy）

例8：二分覆盖问题

问题描述： 二分图是一个无向图，它的n个顶点可分为集合A和集合B，且同一集合中的任意两个顶点在图中无边相连。

$G=(V, E)$

$V=A \cup B \quad A \cap B=\varnothing$

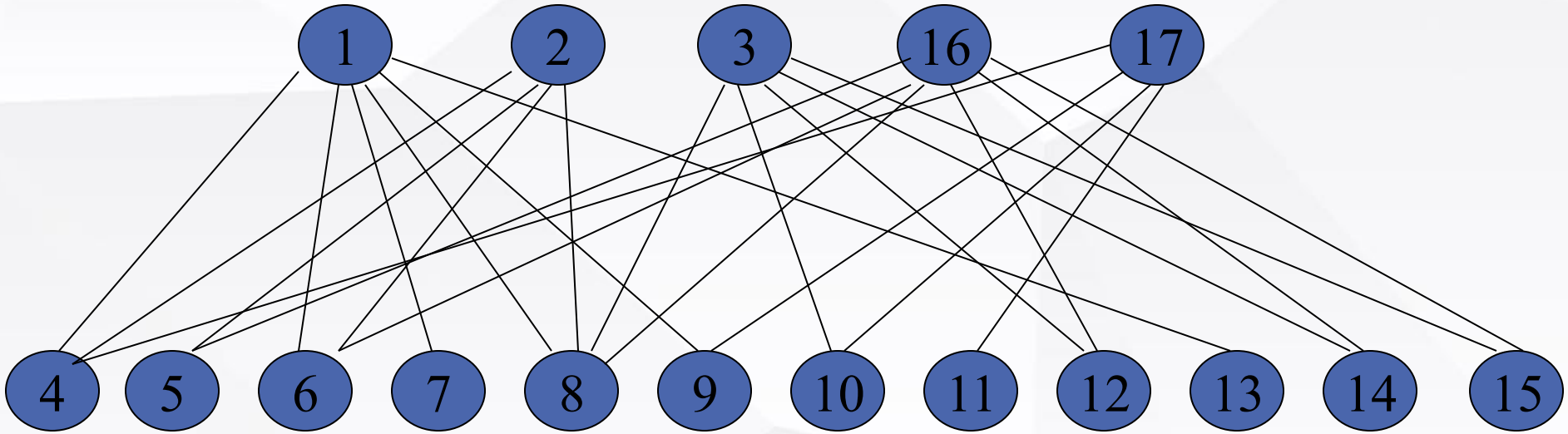
$E=\{(V_i, V_j),\ V_i \in A\ ,\ V_j \in B\}$

覆盖：A'是A的一个子集，它使的B中的每个顶点都至少与A'中的一个顶点相连，称A'是B的覆盖，A'中的顶点个数定义为覆盖的大小。

最小覆盖：A'是覆盖，当它是覆盖B的所有子集中最小的时，称为B的最小覆盖。

会议的最少翻译人员问题，就是二分覆盖的应用！

算法设计策略——贪心策略（Greedy）



覆盖有很多，最小覆盖A′={1,16,17}

贪婪准则：每次从A中选择一个顶点加入覆盖，选择时：选取能覆盖B中还未被覆盖的顶点的数目最多的顶点。可以找到覆盖，但不一定是最小的！！

这是一个NP复杂问题

算法设计策略——贪心策略（Greedy）

类似的集合覆盖问题：给定全集U, 以及一个包含n个集合且这n集合的并集为全集的集合S。集合覆盖问题要找到S的一个最小的子集，使得它们的并集等于全集。

集合覆盖问题为NP-复杂问题。由于集合覆盖与二分覆盖是同一类问题，二分覆盖问题也是NP-复杂问题。因此可能无法找到一个快速的算法来解决它，但是可以利用贪婪算法寻找一种快速启发式方法。

贪婪准则：从集合A中选择一个子集，它能够覆盖B中尽可能多的元素。

算法设计策略——贪心策略（Greedy）

例9：多机调度问题

问题描述： 设有n个独立的作业$\{1,2,\ldots,n\}$，由m台相同的机器进行加工处理。作业i所需要的处理时间为ti。现约定，任何作业可以在任何一台机器上加工处理，但未完工前不允许中断处理。任何作业不能拆分成更小的子作业。

多机调度问题要求给出一种作业调度方案，使所给的n个作业在尽可能短的时间内由m台机器加工处理完成。

这也是一个NP问题，到目前为止还没有一个有效的求解算法。

但是可以采用贪心策略设计出较好的近似算法。

算法设计策略——贪心策略（Greedy）

贪心准则： 最长处理时间作业优先。

当*n*<=*m*时，只要将机器*i*的[0，*ti*]时间区间分配给作业*i*即可。

当*n*>*m*时，首先将*n*个作业依其所需要的处理时间从大到小排序，然后依次顺序将作业分配给空闲的处理机。

例如，设有7个独立作业{1，2，3，4，5，6，7}，由3台机器*M1*，*M2*，*M3*来加工处理。各个作业的处理时间分别为{2,14,4,16,6,5,3}。

按照上面的贪心准则，调度如下：

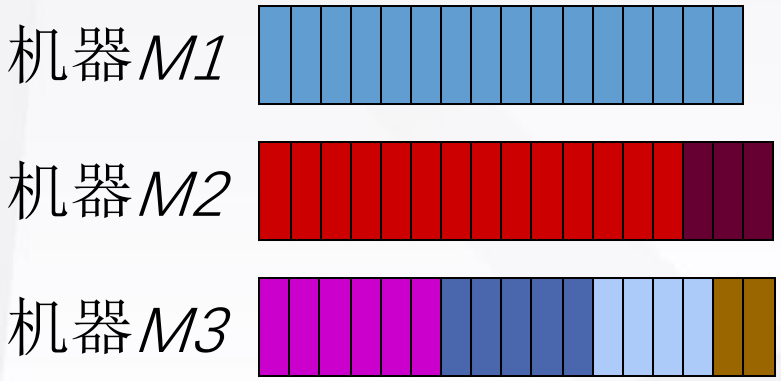
时间最长的第4个作业首先分配给机器*M1*;

时间次长的第2个作业分配给机器*M2*;

时间再次长的第5个作业分配给机器*M3*;

算法设计策略——贪心策略（Greedy）

*M3*机器最早结束，于是，下一个安排的作业是第6个作业；
然后还是*M3*最早结束，于是下一个安排的作业是第3个作业；
然后*M2*最早结束，于是下一个安排的作业是第7个作业；
然后*M3*最早结束，于是下一个安排的作业是第1个作业；
最短时间：17



算法自己写出

贪心策略是求解最优化问题非常简单、有效的算法策略。其关键有两点：

（1）发现贪心准则；

（2）证明正确性；

本 节 结 束！

算法设计策略——贪心策略（Greedy）

本章利用贪心解决了很多最优化问题，这些问题大都代表着一类典型问题：

（1）活动安排问题。请自己列出更复杂的活动安排问题，并探索能不能用贪心策略解决。

（2）最短路径问题。负权值怎么办？

（3）最小生成树问题。有向图能行吗？

（4）背包问题。不可分割背包模型的实例。

（5）调度问题。各种变形问题。

实战—最后一块石头的重量

问题描述：有n（$1 \leqslant n \leqslant 30$）块石头，每块石头的重量都是正整数（重量为1～1000）。每一回合从中选出两块最重的石头，然后将它们一起粉碎。假设石头的重量分别为x和y，且$x \leqslant y$。那么粉碎的可能结果如下：

① 如果 $x==y$，那么两块石头都会被完全粉碎。

② 如果 $x \neq y$，那么重量为 x 的石头将会完全粉碎，而重量为 y 的石头新重量为$y-x$。

最后最多只会剩下一块石头，求此石头的重量，如果没有石头剩下结果为0。要求设计如下函数：

```
class Solution {
public:
    int lastStoneWeight(vector<int>& stones) { }
};
```

解

选石头的过程与构造哈夫曼树的过程类似，只是这里选的是两块最重的石头，用优先队列（大根堆）求解，每次出队两块最重的石头x和y，然后将x-y进队，直到仅有一块石头为止。

```
class Solution {
public:
    int lastStoneWeight(vector<int>& stones)
    {   priority_queue <int> pq;                    //默认为大根堆
        for(int i=0;i<stones.size();i++)
            pq.push(stones[i]);                     //所有石头进队
        int x,y;
        while(!pq.empty())
        {   x=pq.top(); pq.pop();
            if(pq.empty())                          //若x是最后的石头，返回x
            return x;
            y=pq.top();                             //若x不是最后石头，则再出队y
            pq.pop();
            pq.push(x-y);                           //粉碎后新重量为 x-y(x≥y)
        }
        return x;
    }
};
```

7.4.3 实战—赶作业（HDU1789）

问题描述：A有n份作业要做，每份作业有一个最后期限，如果在最后期限后交作业老师就会扣分，现在假设完成每份作业都需要一天。所以A想到了要安排做作业的顺序，把扣分降到最低，请你帮助他。

输入格式：输入包含t个测试用例，第一行是单个整数t。每个测试用例以一个正整数n开头（$1 \leqslant n \leqslant 1000$），表示作业的数量，然后两行，第一行包含$n$个整数，表示作业的截止日期，下一行包含$n$个整数，表示作业的扣分。

输出格式：对于每个测试用例，在一行中输出最少的扣分。

解 属于典型的带惩罚的调度问题，实际上扣分就是惩罚值，最优解就是求最小的总惩罚值。

- 带惩罚的调度问题中，通常假设n个作业加工时间均为一个时间单位，时间用0～maxd的连续整数表示，每个作业有一个截止时间（deadline用时间整数表示），当一个作业在其截止时间之后完成，对应有一个惩罚值（punish）。
- 该问题的最优解是最小总惩罚值。

贪心策略：选择当前惩罚值最大的作业优先加工，按惩罚值递减排序，并且尽可能选择作业截止时间之前最晚的时间加工。按排序后的顺序依次加工。

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
#define MAXN 1010

struct Job                                          //作业的类型
{    int dtime;                                     //截止日期
     int punish;                                    //扣分
     bool operator<(const Job& b) const
     {
          return punish>b.punish;                   //按punish递减排序
     }
};
int n;
Job a[MAXN];
```

```
int greedly(int maxd)                          //贪心算法
{   vector<bool> days(maxd,false);
    sort(a,a+n);                               //排序
    int ans=0;
    for(int i=0;i<n;i++)
    {   int j=a[i].dtime;
        for(;j>0;j--)                          //查找截止日期之前（含）的空时间
        {   if(days[j]==false)                 //找到空时间
            {   days[j]=true;
                break;
            }
        }
        if(j==0)                               //没有找到空时间
            ans+=a[i].punish;                  //累计扣分
    }
    return ans;
}
```

```
int main()
{    int t;
     scanf("%d",&t);
     while(t--)
     {    int maxd=0;
          scanf("%d",&n);
          for(int i=0;i<n;i++)
          {    scanf("%d",&a[i].dtime);
               maxd=max(maxd,a[i].dtime);
          }
          for(int i=0;i<n;i++)
               scanf("%d",&a[i].punish);
          printf("%d\n",greedly(maxd));
     }
     return 0;
}
```

- 上述程序提交时通过，执行时间为46ms，内存消耗为1748KB。
- greedly算法的时间复杂度为O(*n**maxd)，可以采用并查集改进时间性能。
- 设置并查集数组parent，parent[*i*]表示第*i*天的前一个空时间（含第*i*天），初始时置parent[*i*]=*i*，当第*i*天被占用时合并操作是置parent[*i*]=*i*-1（可以理解为第*i*天被占用，前面的空时间与第*i*-1天前面的空时间相同），由于这里的合并有方向性，所以不用rnk数组。

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
#define MAXN 1010
vector<int> parent;                          //并查集存储结构
vector<int> rnk;                                 //存储结点的秩(近似于高度)

int Find(vector<int>& parent,int x)          //递归算法：查找x结点的根结点
{    if (x!=parent[x])
          parent[x]=Find(parent,parent[x]);      //路径压缩
     return parent[x];
}
```

```
struct Job                                          //作业的类型
{    int dtime;                                     //截止日期
     int punish;                                    //扣分
     bool operator<(const Job& b) const
     {
          return punish>b.punish;                   //按punish递减排序
     }
};
int n;
Job a[MAXN];
```

```
int greedly(int maxd)                          //贪心算法
{   parent.resize(maxd);
    for(int i=0;i<=maxd;i++)                   //查集初始化
        parent[i]=i;
    sort(a,a+n);                               //排序
    int ans=0;
    for(int i=0;i<n;i++)
    {   int day=Find(parent,a[i].dtime);       //查找截止日期之前的空时间
        if(day>0)                              //找到空时间
            parent[day]=day-1;                 //合并
        else
            ans+=a[i].punish;                  //累计扣分
    }
    return ans;
}
```

```
int main()
{    int t;
     scanf("%d",&t);
     while(t--)
     {    int maxd=0;
          scanf("%d",&n);
          for(int i=0;i<n;i++)
          {    scanf("%d",&a[i].dtime);
               maxd=max(maxd,a[i].dtime);
          }
          for(int i=0;i<n;i++)
               scanf("%d",&a[i].punish);
          printf("%d\n",greedly(maxd));
     }
     return 0;
}
```

上述程序提交时通过，执行时间为31ms，内存消耗为1748KB。

46ms